静的バイナリ解析によるML-KEM・ML-DSA実装検出

Hendrik De Coster[†1] 山室 宏貴[1,a)] 福島 和英[1]

概要：量子コンピュータの実用化に伴う既存の暗号技術の危殆化に備え，耐量子計算機暗号 (PQC) への移行が求められている．PQC 移行では，暗号技術の利用状況を把握するための暗号インベントリの整備が重要である．実運用環境では，検出に利用可能な情報ソースがバイナリコードに限られる場合もある．しかし，既存のバイナリ解析による検出手法は AES や SHA などの従来暗号を対象としており，PQC 実装を直接検出することを目的とした手法は十分に検討されていない．
本研究では，バイナリコードから ML-KEM と ML-DSA 実装を検出するための静的バイナリ解析手法を提案する．提案手法は識別子，定数値，コード構造という 3 種類の情報を活用する．識別子と定数値はシグネチャ照合により検出し，定数値については機械語命令中の即値も利用する．コード構造は機械語命令や制御フローグラフから抽出した特徴量を用いた機械学習により検出する．これらは異なる情報を利用するため，ストリップや最適化で一部の情報が失われる場合にも相補的な検出が可能である．このうち，コード構造に基づく手法を Leave-One-Group-Out 交差検証により評価した結果，マクロ平均 F_2 スコアは 0.801 であり，学習に含まれないライブラリに対しても ML-KEM と ML-DSA を検出できることを確認した．

キーワード：耐量子計算機暗号移行, 暗号技術検出, 静的バイナリ解析, ML-KEM, ML-DSA

Static Binary Analysis for Discovering ML-KEM and ML-DSA

Hendrik De Coster[†1] Hiroki Yamamuro[1,a)] Kazuhide Fukushima[1]

Abstract: The advent of practical quantum computers is expected to compromise existing cryptographic algorithms, driving the transition to post-quantum cryptography(PQC). Such a transition depends on a cryptographic inventory of the cryptographic algorithms in use. In operational environments, information sources available for such identification may be limited to binary code. However, existing binary-analysis-based discovery approaches have mainly targeted conventional cryptographic algorithms, such as AES and SHA, and methods aimed at directly discovering PQC implementations have not been sufficiently explored. This paper proposes a static binary analysis method for discovering ML-KEM and ML-DSA implementations from binary code. The proposed method exploits three types of information: identifiers, constant values, and code structure. Identifiers and constant values are detected through signature matching, and constant values are additionally detected from immediate operands in machine instructions. Code structure is analyzed using machine learning with features extracted from machine instructions and control-flow graphs. Because these techniques rely on different information, they enable complementary detection even when stripping or compiler optimization removes some information. Among these, the code-structure-based method is evaluated under Leave-One-Group-Out cross-validation, achieving a mean macro-F_2 score of 0.801 and confirming that ML-KEM and ML-DSA can be discovered even in libraries unseen during training.

Keywords: PQC Transition, Cryptographic Discovery, Static Binary Analysis, ML-KEM, ML-DSA

[1] KDDI 総合研究所, 埼玉県ふじみ野市大原 2 丁目 1 番 15 号, KDDI Research, Inc., 2-1-15 Ohara, Fujimino-shi, Saitama, Japan
[†1] 現在，Ghent University, Ghent, Belgium
[a)] hi-yamamuro@kddi.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

1.1 背景

Shor [1] の量子アルゴリズムにより RSA 暗号や楕円曲線暗号など幅広く利用されている公開鍵暗号方式が危殆化するため，量子コンピュータ実用化に備え，耐量子計算機暗号 (PQC) に移行することが求められる．PQC は米国国立標準技術研究所 (NIST) により標準化が進められており，2022 年に ML-KEM [2]，ML-DSA [3]，SLH-DSA [4]，FN-DSA [5], 2025 年に HQC [6] が選定された．さらに，追加電子署名プロジェクト [7] は 2026 年 5 月に第 3 ラウンドへ移行している．

PQC 移行に向けた取り組みが世界各国で進められており，米国，英国，EU などではロードマップ・タイムライン [8], [9], [10] が策定されている．日本では，2025 年にロードマップの骨子 [11] が公表され，2026 年度中に完成版が策定される予定となっている．PQC 移行の第一歩として，利用中の暗号方式やパラメータセットを一覧化する暗号インベントリの整備が重要であり，標準的な記述形式の一つに Cryptographic Bill of Materials(CBOM) [12] がある．暗号インベントリの整備に必要な情報の取得には，使用されている暗号技術を検出する手法が求められる．検出に利用可能な情報ソースとして，山室ら [13] が整理しているように，ソースコード，バイナリコード，セキュリティプロトコルの設定ファイルなどが挙げられる．実運用では，ソースコードや設定ファイルを取得できず，バイナリコードのみが利用可能な場合もあるため，バイナリコードを対象とした自動検出手法が必要となる．

PQC 移行を目的とした QED [14] や QED-Lite [15] では，量子脆弱な実行ファイルの特定や暗号ライブラリの PQC 対応状況の判定が行われている．また，リバースエンジニアリングやマルウェア解析の分野では，バイナリコードから暗号技術を検出する様々な手法 [16], [17], [18], [19] が提案されているが，主な対象は暗号化方式やハッシュ関数などの従来の暗号技術である．これらの既存研究はいずれも，バイナリコード中の PQC 実装そのものを直接検出することを目的としていない．

PQC 移行では，量子脆弱な暗号技術の特定に加え，導入済み PQC の確認，新たな脆弱性に伴う再移行に備えて PQC 実装そのものを把握することも重要である．実際，追加電子署名プロジェクトの候補であった HAWK [20] は，2026 年に鍵回復攻撃 [21] が示され，著者らによって標準化プロセスから取り下げられた．NIST 標準の ML-KEM と ML-DSA は特に主要な移行先であり，ML-KEM は Web ブラウザや OS，クラウドサービスなどで導入が進み，ML-DSA もクラウドサービスや証明局基盤などで対応が始まっている．したがって，ML-KEM と ML-DSA は PQC の導入状況を把握する上で重要な検出対象である．

1.2 貢献

本研究は，バイナリコード中に含まれる ML-KEM と ML-DSA の実装を静的に検出する手法を提案する．提案手法では，コンパイル後のバイナリコードから取得可能な識別子，定数値，コード構造の 3 種類の情報に着目し，それぞれに基づく検出手法を相補的に組み合わせる．識別子とデータ領域に配置された定数列の検出には，特徴的なパターンをシグネチャとしてルール化し，バイナリコードと照合するパターンマッチングツールである YARA [22] を用いる．主な貢献は以下の 3 点である．

(1) **識別子を利用した YARA ルールによる検出**
ML-KEM と ML-DSA の各パラメータセットに対応する Object Identifier(OID) の DER(Distinguished Encoding Rules) 符号化バイト列に加え，API 名，シンボル名，アルゴリズム名などを YARA ルールとして定義する．識別子から方式を識別でき，パラメータセット固有の OID や名称が残存する場合にはパラメータセットまで識別できる．

(2) **定数値を利用した YARA ルールと命令中の即値による検出**
ML-KEM と ML-DSA の Number Theoretic Transform(NTT) や剰余演算で用いられる特徴的な定数列を YARA ルールとして定義する．また，ベクトル化された実装では法やその逆元が複数要素に並べられた定数ベクトルとして配置されるため，これらも YARA ルールとして定義する．さらに，逆アセンブルした命令から命令中に直接埋め込まれた定数値であるイミディエイト (即値) を抽出し，各方式の法や剰余演算に関連する特徴的な定数値を検出する．複数の即値の共起や出現状況を考慮することで，偶発的な一致による誤検出を抑制する．これらの定数値は各方式のパラメータセット間で共通するため，方式は識別できるがパラメータセットまでは識別できない．

(3) **コード構造に基づく機械学習による検出**
逆アセンブルした関数から制御フローグラフ (CFG) を構築し，命令の出現傾向や制御フローなど ML-KEM と ML-DSA の演算構造を表す特徴量を抽出する．これらの特徴量を用いて，第 1 段階で ML-KEM または ML-DSA の候補を背景コードから抽出し，第 2 段階で ML-KEM，ML-DSA および背景コードを識別する 2 段階の XGBoost 分類器 [23] を構築する．複数の暗号ライブラリを 1 つずつ学習データから除外し，テストデータとする Leave-One-Group-Out 交差検証 (LOGO-CV) を行った結果，マクロ平均 F_2 スコアは 0.801 であり，学習に含まれないライブラリに対しても検出できることを示す．

© 2026 Information Processing Society of Japan

1.3 関連研究

本節では，バイナリコードから暗号技術を検出する先行研究と PQC の利用状況や方式を識別する先行研究を紹介する．

PQC 移行を支援するためのバイナリ解析手法として，QED [14] と QED-Lite [15] が提案されている．

QED は量子脆弱な暗号技術を利用する実行ファイルを特定するため，実行ファイルと暗号ライブラリの依存関係，呼び出される暗号 API，量子脆弱な暗号 API へ至る静的な呼出し経路を段階的に解析する．

QED-Lite はより軽量な解析を目的として，実行ファイルが依存する暗号ライブラリとそのバージョンを特定し，ライブラリのバージョンに基づいて PQC への対応状況を判定する．

リバースエンジニアリングやマルウェア解析では，バイナリコード中で使用される暗号技術を特定するため，暗号関数を検出・識別する様々な手法が提案されている．

Where's Crypto? [16] は対象関数をシンボリック実行しながらデータフローグラフ (DFG) を構築し，暗号プリミティブごとに定義したグラフパターンとの部分グラフ同型性を判定することでバイナリコード中の暗号プリミティブを検出・分類する．

FoC [17] は大規模言語モデルを用いてストリップされたバイナリの疑似コードから関数名や処理内容の自然言語要約を生成する FoC-BinLLM と，その意味表現に制御フローグラフの構造情報や暗号関連の特徴量を組み合わせてバイナリコードの類似度を解析する FoC-Sim から構成され，既知の暗号実装と類似する関数を識別する．

Mnemocrypt [18] は逆アセンブルした関数から，機械語命令の種類を表すニーモニックの出現割合や命令の並びに関する統計量を抽出する．さらに，基本ブロック数やループ数などの制御フロー構造に関する特徴量を組み合わせ，Random Forest 分類器を用いて暗号関数か否かを判定する．

BinCrypto [19] は対象関数と既知の暗号関数に同一の入力を与え，すべての基本ブロックを一度ずつ処理するエミュレーションによって入出力値を特徴量として抽出する．両者から得られた特徴量の類似度を計算することで，対象関数が既知の暗号アルゴリズムを実装しているかを識別する．

PQC を対象とした識別手法として，Mallick ら [24] はバイナリコードではなく，プロセスの実行時に観測される CPU サイクル数やメモリ使用量を特徴量として機械学習による分類を行い，従来の暗号方式と PQC の識別や PQC アルゴリズムおよびライブラリ実装の分類を行う．また，TLS，SSH，QUIC などの通信についてはハンドシェイクメッセージから得られる鍵長などの情報を用いて従来方式と PQC を用いた鍵交換を識別する．

1.4 構成

2 節では，ML-KEM と ML-DSA の算術構造を整理するとともに，コンパイル後のバイナリコードに残存する情報とストリップやコンパイラ最適化による影響について述べる．3 節では，ML-KEM と ML-DSA の検出に利用する識別子を整理し，YARA を用いて検出する方法について述べる．4 節では，NTT や剰余演算に関連する定数値を整理し，YARA による定数列の検出と機械語命令中の即値を用いた検出方法について述べる．5 節では，機械語命令や CFG から抽出した特徴量を用いた ML-KEM と ML-DSA の検出手法について述べ，LOGO-CV による評価結果と特徴量の重要度について示す．6 節では，本研究のまとめについて述べる．

2. ML-KEM と ML-DSA のバイナリコード上の特徴

本節では，ML-KEM と ML-DSA の実装をバイナリコードから検出する際に利用できる特徴について述べる．初めに両方式に共通する NTT や剰余演算などの算術構造を整理し，次にコンパイルした後のバイナリコードに残存する情報とストリップやコンパイラ最適化による影響について述べる．

2.1 算術構造

ML-KEM と ML-DSA はモジュール格子に基づく暗号方式であり，多項式環 $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ 上の演算を主要な処理として含む．多項式乗算を効率的に行うために NTT が用いられ，バタフライ演算によって乗算と剰余演算が繰り返し行われる．ML-KEM では $q = 3329$，ML-DSA では $q = 8380417$ を法として用いる．ML-KEM の法は 16 ビット符号付き整数の範囲に収まるのに対し，ML-DSA の法はこの範囲を超えるため，Single Instruction, Multiple Data(SIMD) を用いた実装ではそれぞれ 16 ビット整数単位と 32 ビット整数単位のベクトル演算が用いられる場合があり，命令列に差異が生じ得る．また，NTT では原始根の冪に由来する値が定数列として保持される場合があるほか，Montgomery reduction や Barrett reduction などの剰余演算では法 q に依存する定数が用いられる場合がある．

両方式では行列生成時に棄却サンプリングが用いられる．さらに，ML-DSA では，鍵生成時の小さい係数を持つ秘密ベクトルの生成，署名候補の棄却・再生成，鍵と署名をコンパクトに表現するためのビットパッキング処理が行われる．

2.2 コンパイル後に残存する情報

ソースコードをコンパイルすると，関数名やソースコード上の構造などの情報の一部は失われるが，実装に由来する情報は異なる形でバイナリコード中に残存する．スト

© 2026 Information Processing Society of Japan

リップを行うと，シンボル情報が削除または縮減され，関数名やシンボル名などの情報の多くが失われる．ただし，OID やアルゴリズム名などがデータとしてバイナリに含まれている場合にはストリップ後にも残存することがある．また，NTT で用いられる定数列や法，剰余演算に関連する定数値はシンボル名に依存しないため，読み取り専用データ領域や機械語命令中の即値として残存し，NTT や剰余演算などの処理自体も，乗算，加減算，シフト，ループ，分岐などの機械語命令や制御フローとして現れる．

一方，インライン展開，ループ展開，ベクトル化などのコンパイラ最適化によって，関数構造やループ構造が変化し，生成される命令列や制御フローも変化する．定数値も命令中の即値として現れる場合やデータ領域に配置される場合など，実装や最適化によって表現が異なる．特に AVX2 などのベクトル拡張命令を用いた実装では，複数のスカラー演算が SIMD 命令によってまとめて処理されるほか，法やその逆元などが定数ベクトルとして配置される場合がある．さらに，ラッパ関数や間接呼出しを伴う抽象化層を持つ実装では，暗号処理に関連する命令が複数の関数に分散する場合があり，関数単位で観測される命令パターンも実装方式やコンパイラ最適化によって異なる．

このように，コンパイル後のバイナリコードには，ストリップやコンパイラ最適化の影響を受けながらも，ML-KEM と ML-DSA の実装に由来する識別子，定数値，コード構造などの情報が残存する．

3. 識別子に基づく検出

本節では，バイナリコード中に残存する識別子を利用した ML-KEM と ML-DSA の検出手法について述べる．初めに検出に利用可能な識別子について整理し，次に YARA を用いた検出方法について述べる．

3.1 利用する識別子

検出には OID，API 名やシンボル名，アルゴリズム名など，ML-KEM と ML-DSA に関連する識別子を利用する．OID は，暗号方式やパラメータセットを識別するために割り当てられる識別子であり，Abstract Syntax Notation One(ASN.1) で使用される場合には DER 符号化されたバイト列として現れる．ML-KEM と ML-DSA では各パラメータセットに固有の OID が割り当てられているため，対応するバイト列を検出することでパラメータセットまで識別できる．表 1 に，これらの OID と対応する DER 符号化バイト列を示す．

API 名やシンボル名については，各暗号ライブラリで ML-KEM と ML-DSA に関連して用いられる名称を利用する．例えば，OpenSSL [25] では，`ossl_ml_kem`，`ossl_ml_dsa` などの名称が用いられ，方式のみを示す名称とパラメータセットまで示す名称が存在する．また，`mlkem512`，`mldsa44` などアルゴリズム名を含む文字列も識別子として利用する．これらの文字列は，アルゴリズム名の一覧や設定情報，エラーメッセージなどに含まれる場合がある．

表 1: ML-KEM/ML-DSA の OID と DER 表現

パラメータセット	OID・DER 符号化バイト列
ML-KEM-{512,768,1024}	2.16.840.1.101.3.4.4.{1,2,3}
	06 09 60 86 48 01 65 03 04 04 {01,02,03}
ML-DSA-{44,65,87}	2.16.840.1.101.3.4.3.{17,18,19}
	06 09 60 86 48 01 65 03 04 03 {11,12,13}

3.2 検出方法

識別子の検出には YARA を用いる．OID については表 1 に示した DER 符号化バイト列を，API 名，シンボル名，アルゴリズム名については対応する文字列を YARA ルールとして定義し，対象バイナリとの一致を検出する．検出された識別子には対応する方式とパラメータセットの情報を関連付ける．パラメータセット固有の OID や名称が一致した場合には ML-KEM-512 や ML-DSA-44 のようにパラメータセットまで識別する．一方，`ossl_ml_kem` や `mldsa` のように方式のみを表す識別子が一致した場合には ML-KEM または ML-DSA の方式までを検出結果とする．

4. 定数値に基づく検出

本節では，バイナリコード中に残存する定数値を利用した ML-KEM と ML-DSA の検出手法について述べる．初めに，検出に利用する定数値について整理し，次に定数列と機械語命令中の即値を用いた検出方法について述べる．

4.1 利用する定数値

検出には NTT で用いられる定数列と法 q や Montgomery reduction，Barrett reduction などの剰余演算に関連する方式固有の定数値を利用する．ML-KEM では $q = 3329$，ML-DSA では $q = 8380417$ を法として用いる．また，Montgomery reduction や Barrett reduction ではそれぞれの法から導出される定数値が使用される．表 2 に，検出に利用する代表的な定数値を示す．

表 2: ML-KEM/ML-DSA の代表的な定数値

方式	定数値	用途・定義式
ML-KEM	3329	法 q
	62209	Montgomery reduction：$q^{-1} \bmod 2^{16}$
	20159	Barrett reduction：$\lfloor 2^{26}/q + 1/2 \rfloor$
ML-DSA	8380417	法 q
	58728449	Montgomery reduction：$q^{-1} \bmod 2^{32}$
	4236238847	Montgomery reduction：$-q^{-1} \bmod 2^{32}$

また，AVX2 を用いてベクトル化された実装では，同一の法やその逆元などを複数要素に並べた定数ベクトルが用

© 2026 Information Processing Society of Japan

いられる場合があり，このような定数ベクトルも NTT 関連の定数列と同様に検出に利用する．

4.2 検出方法

定数値の検出では，データ領域に配置された定数列と実行コード中の機械語命令に現れる即値をそれぞれ検出する．NTT に関連する定数列については，コンパイル時に値が確定する読み取り専用データが配置される `.rodata` などのデータ領域を対象として，原始根の冪からなる特徴的な定数列を表現ごとに YARA ルールとして定義して検出する．また，AVX2 を用いたベクトル化実装については法やその逆元などが 8 個または 16 個連続して配置された定数ベクトルを YARA ルールとして定義し，同様に検出する．初期化済みの読み書き可能データを配置する `.data` などに定数が配置される場合もあるため，これらの静的データ領域も解析対象とする．

一方，機械語命令中に直接埋め込まれた定数値については，関数本体の機械語命令列が配置される `.text` などの実行コード領域を Capstone [26] で逆アセンブルし，各命令から即値を抽出する．抽出した即値について，表 2 に示した法や剰余演算に関連する定数値と一致を調べる．このとき，任意のバイト列から数値を探索するのではなく，有効な機械語命令の即値として現れる値のみを対象とすることでデータ領域や命令途中のバイト列を誤って即値として検出することを防ぐ．また，法の値は一般の数値処理にも偶然現れる可能性があるため，単一の定数値との一致のみで判定せず，法の反復出現と複数種類の剰余演算に関連する定数値の共起を考慮して検出する．

ML-KEM と ML-DSA では各パラメータセットに共通して $q = 3329$ と $q = 8380417$ がそれぞれ用いられるため，これらの定数値から方式を識別することはできるが，パラメータセットまでは識別できない．

5. コード構造に基づく検出

本節では，機械語命令や制御フローなどのコード構造から抽出した特徴量を用いた ML-KEM と ML-DSA の検出手法とその検出性能について述べる．3，4 節で述べた識別子や定数値は，バイナリコード中に残存している場合には ML-KEM と ML-DSA を検出するための直接的な手掛かりとなる．一方，ストリップによってシンボル情報が削除されたり，コンパイラ最適化によって定数値が変形または消失したりすることでこれらの情報のみでは検出が困難となる場合がある．

5.1 利用する特徴量

コード構造に基づく検出では対象関数を逆アセンブルして得られる機械語命令と CFG から特徴量を抽出する．特徴量には命令の種類とその出現割合，基本ブロック内の命令分布，ループや分岐などの制御フロー構造，メモリアクセスの規則性などを用いる．

ML-KEM と ML-DSA の NTT では乗算と剰余演算が繰り返されるため，乗算命令やシフト命令が特定の基本ブロックに集中して出現する．そこで，関数全体と基本ブロックごとの乗算・シフト命令の出現数と命令数で正規化した密度を特徴量として用いる．NTT のバタフライ演算に由来するループ構造については，ループを構成する基本ブロック数の割合，後退辺密度，強連結成分に基づくループの深さ，循環的複雑度などの CFG に関する特徴量で捉える．加えて，NTT における規則的なメモリアクセスを捉えるため，機械語命令のオペランドに現れる変位量の規則性も特徴量に含める．

ML-KEM と ML-DSA では法 q の大きさが異なり，SIMD を用いた実装ではそれぞれ 16 ビットと 32 ビット単位のベクトル演算が用いられる場合がある．この違いを捉えるため，ベクトル命令における 16 ビットと 32 ビット単位の演算の出現状況を両方式の識別に利用する．さらに，法 $q = 3329$ と $q = 8380417$ の出現数，Montgomery reduction と Barrett reduction に関連する定数値の出現密度，ML-DSA のビットパッキングや署名生成時の棄却処理に由来するシフト・比較命令の密度も補助的な特徴量として用いる．他の格子暗号方式や符号ベース暗号方式との誤検出を抑制するため，浮動小数点演算の出現割合や論理演算に対する算術演算の比率も特徴量に含める．以上を合わせて，1 関数あたり 121 次元の特徴ベクトルを構成する．

5.2 検出方法

コード構造に基づく検出は関数単位で行うため，初めにバイナリコード中で各関数の範囲を特定する．シンボル情報が残存している場合にはシンボルテーブルから関数の開始位置と終了位置を取得する．ストリップによってシンボル情報が削除されている場合には例外処理などでスタックを巻き戻すために用いられるアンワインド情報を利用する．具体的には，ELF では `.eh_frame`，Windows PE x64 では `.pdata` に含まれる情報から取得可能な範囲で関数の開始位置と終了位置を特定する．取得した各関数を逆アセンブルし，分岐命令の直後や分岐先を基準として基本ブロックに分割する．基本ブロック間の遷移を有向辺として CFG を構築し，5.1 節で述べた特徴量を抽出する．

抽出した特徴ベクトルは 2 段構成の XGBoost 分類器に入力する．第 1 段の分類器 (Stage 1) は対象関数が ML-KEM または ML-DSA に関連する候補であるか，それ以外の背景コードであるかを判定するゲートキーパーであり，乗算やシフトの密度，ループ構造などの構造的特徴を中心に判定を行う．ただし，$q = 3329$ または $q = 8380417$ が有効な機械語命令の即値として現れる関数については構造的特徴のみを理由として Stage 1 で棄却されないようにする．具

© 2026 Information Processing Society of Japan

体的には，これらの法を含む関数では Stage 1 の判定閾値を 0 とし，Stage 2 へ引き渡す．これにより，コード構造が典型的でない関数についても法の値が明示的に現れる場合には Stage 1 における見逃しを抑制する．

Stage 1 を通過した関数は，第 2 段の分類器 (Stage 2) によって Negative，ML-KEM，ML-DSA の 3 クラスに分類される．Stage 2 ではコード構造に加えて，法や剰余演算に関連する定数値の出現状況を利用することで ML-KEM と ML-DSA を識別する．ML-KEM または ML-DSA の出力確率がいずれも所定の閾値を下回る場合には Negative として棄却する．

なお，手書きの AVX2 実装を学習対象に含めると，学習結果が特定のベクトル命令のパターンに依存し，未知の実装に対する汎化性能が低下する可能性があるため，これらを機械学習の学習対象から除外する．これらの実装については，4 節で述べた定数ベクトルに基づく検出を用いる．一方，コンパイラによって自動的にベクトル化されたコードについては 5.1 節で述べたベクトル命令に関する特徴量を利用する．

5.3 評価方法

コード構造に基づく検出手法の汎化性能を評価するため，ML-KEM または ML-DSA の実装を含む複数の暗号ライブラリと背景コードを用いて評価用データセットを構築した．背景コードには，圧縮処理，線形代数，FFT などの信号処理，データ解析，多倍長整数演算などを行う汎用ライブラリを用いた．これらには乗算やループを多く含み，NTT をはじめとする ML-KEM と ML-DSA の算術処理と表面的に類似した命令構成を持つコードも含まれ，これにより類似した算術的特徴を持つ一般コードに対する誤検出も含めて識別性能を評価した．

各ライブラリを x86-64 向けに GCC，Clang，MSVC でコンパイルし，`-O0`，`/Od` や `-O2`，`/O2` など複数の最適化オプションによって生成されたバイナリコードを用いた．構築したデータセットには合計 106770 関数が含まれ，各関数から 121 次元の特徴量を抽出した．なお，背景コードはすべての fold で共通して評価するため，全 fold の予測結果を集約した際の予測件数はデータセットに含まれる実際の関数数とは一致しない．暗号ライブラリには暗号演算そのものを行わないラッパ関数や補助関数も含まれるため，これらをすべて正例とはせず，ML-KEM と ML-DSA の算術処理に対応する関数のみを学習対象の正例として用いた．また，5.2 節で述べた理由により手書きの AVX2 実装は学習対象には含めなかった．

未知のライブラリに対する汎化性能を評価するため，各 fold で 1 つの暗号ライブラリ全体をテストデータとして退避し，残りの暗号ライブラリで分類器を学習する LOGO-CV を行った．これにより，テスト対象となるライブラリに固有のコードの記述方法は学習時には使用されない．背景コードを構成する汎用ライブラリをすべての fold のテストデータに共通して含め，ML-KEM と ML-DSA 以外の関数を適切に棄却できるかをあわせて評価した．この操作をすべての暗号ライブラリについて繰り返し，各ライブラリを学習に用いない条件での検出性能を評価した．

評価指標には，適合率 P と再現率 R に基づくマクロ F_2 スコアを用いる．F_2 スコアは

$$F_2 = \frac{5PR}{4P + R}$$

で表される．F_2 は適合率よりも再現率を重視する指標であり，見逃しを誤検出よりも重く評価するために採用した．各 fold ではテストデータ中に正解ラベルが存在するクラスについて F_2 スコアを算出し，そのマクロ平均を評価値とした．また，コンパイラや最適化レベルによる検出性能への影響を確認するため，コンパイラごとおよび最適化レベルごとにマクロ F_2 スコアを算出した．

5.4 検出性能

各 fold におけるマクロ F_2 スコアは，平均 0.801，標準偏差 0.103 であった．図 1 に，全 fold の予測結果を集約した 3 クラスの混同行列を示す．背景コードの 99.9% が

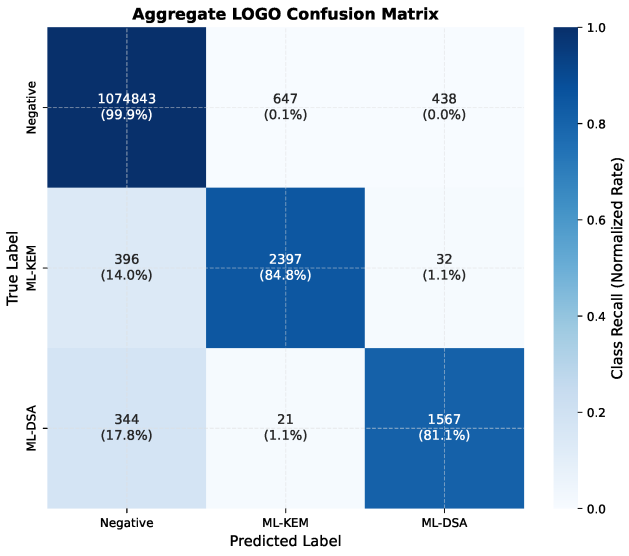


図 1: 全 fold の予測結果を集約した 3 クラス混同行列．

Negative として正しく分類されており，背景コードに対する誤検出は極めて少ない．ML-KEM は 2397 件，ML-DSA は 1567 件がそれぞれ正しく分類され，方式間の取り違えは合計 53 件にとどまった．全 fold を通じて生じた誤りのうち，方式間の取り違えは 2.8% に過ぎず，残りは背景コードを PQC と誤検出した誤り (57.8%) および PQC を背景コードとして見逃した誤り (39.4%) であった．ML-KEM と ML-DSA の方式間の識別は高い精度で行えており，主

© 2026 Information Processing Society of Japan

な誤りは PQC と背景コードとの識別において生じていることが分かる．

仮想関数やラッパ関数などの抽象化層を持つ実装や，法の値を直接命令中に埋め込まないポータブルな実装では，命令が複数の関数に分散し算術的特徴の密度が低下するため，検出性能が相対的に低下する傾向が見られた．このほか，複数の実装を集約したライブラリでは他の格子暗号方式や符号ベース暗号方式など類似した演算構造を持つ実装に起因する誤検出も確認された．

コンパイラや最適化レベルによる傾向を確認したところ，最適化なしのコード (`-O0`，`/Od`) よりも最適化されたコード (`-O2`，`-O3`，`-Ofast`，`/Ox`) で高い検出性能を示す傾向が確認された．最適化なしのコードではスタックへの退避処理や関数呼び出しのインライン化不足により乗算・シフトの命令密度が低下する一方，最適化によって演算が集約されることで構造的特徴が現れやすくなり，検出性能が向上したと考えられる．

5.5 特徴量重要度

分類器がどの特徴量に基づいて判定しているかを確認するため，XGBoost における特徴量の gain を用いて分析を行った．gain は決定木の分割に各特徴量がどの程度寄与したかを表す指標である．図 2 に，Stage 1・Stage 2 それぞれにおける gain 上位の特徴量を示す．

図 2a の Stage 1 では，基本ブロック内の乗算密度を表す `bb_mean_unified_mul` や乗算数を表す `total_mul`，シフト密度を表す `density_sh`，ループ内の基本ブロックの割合を表す `loop_block_density` などが高い gain を示した．これらは NTT のバタフライ演算における乗算・剰余演算の繰り返しやループ構造に対応する特徴量である．一方，$q = 3329$ や $q = 8380417$ の出現数は Stage 1 の上位特徴量には現れなかった．Stage 1 は特定の定数値ではなく，主にコード構造に基づいて ML-KEM または ML-DSA の候補を背景コードから分離している．

図 2b の Stage 2 では，$q = 3329$ の出現数を表す `count_constant_3329` と $q = 8380417$ の出現数を表す `count_constant_8380417` が特に高い gain を示した．また，ML-KEM と ML-DSA それぞれの剰余演算に関連する定数値の密度 (`mlkem_reduction_constant_clean_density`, `mldsa_reduction_constant_clean_density`) も上位に現れた．Stage 1 が乗算・シフト・ループなどのコード構造を中心に ML-KEM または ML-DSA の候補を背景コードから抽出するのに対し，Stage 2 では方式固有の法や剰余演算に関連する特徴量が ML-KEM と ML-DSA の識別に大きく寄与している．

6. おわりに

本研究では，バイナリコード中の ML-KEM と ML-DSA 実装を静的に検出するため，識別子，定数値，コード構造の 3 種類の情報に基づく検出手法を提案した．識別子と定数値については YARA によるシグネチャ照合を用い，定数値についてはさらに機械語命令中の即値を利用し，コード構造については機械語命令と CFG から抽出した特徴量を用いる 2 段階の XGBoost 分類器により検出する．これらは異なる情報を利用するため，ストリップやコンパイラ最適化によって一部の情報が失われる場合にも相補的な検出が可能である．

コード構造に基づく手法を複数の暗号ライブラリを用いた LOGO-CV で評価した結果，マクロ平均 F_2 スコアは 0.801 となり，学習に含まれないライブラリに対しても ML-KEM と ML-DSA を検出できることを確認した．一方，抽象化層を持つ実装や暗号処理に関連する命令が複数の関数に分散する実装では検出性能が低下する場合がある．今後はこのような実装に対する検出性能の向上とより多様なライブラリやアーキテクチャを対象とした評価を進める．

参考文献

[1] Shor, P. W.: Algorithms for Quantum Computation: Discrete Logarithms and Factoring, *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, IEEE Computer Society, pp. 124–134 (online), DOI: 10.1109/SFCS.1994.365700 (1994).

[2] NIST: Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 203 (2024). `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf`.

[3] NIST: Module-Lattice-Based Digital Signature Standard, FIPS 204 (2024). `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf`.

[4] NIST: Stateless Hash-Based Digital Signature Standard, FIPS 205 (2024). `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf`.

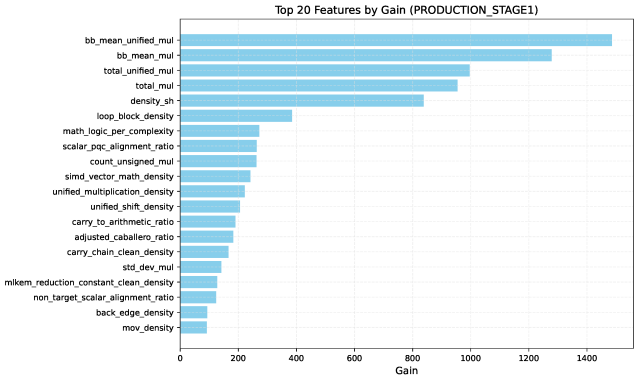
[5] Fouque, P.-A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W. and Zhang, Z.: Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU (2020). `https://falcon-sign.info/falcon.pdf`.

[6] Melchor, C. A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Bos, J., Deneuville, J.-C., Dion, A., Gaborit, P., Lacan, J., Persichetti, E., Robert, J.-M., Véron, P. and Zémor, G.: Hamming Quasi-Cyclic (HQC) (2025). `https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf`.

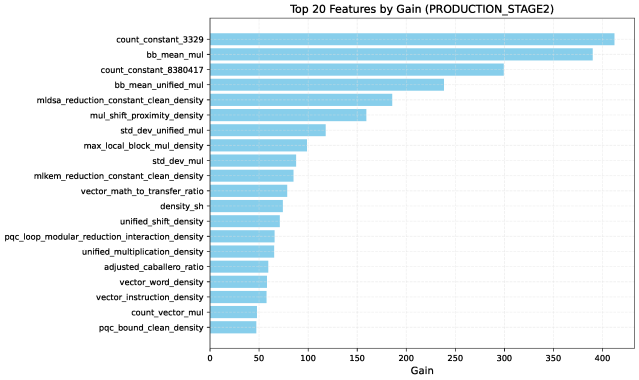
[7] NIST: Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process (2022). `https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf`.

[8] Office of Management and Budget: Execution of the Migration to Post-Quantum

© 2026 Information Processing Society of Japan



(a) Stage 1



(b) Stage 2

図 2: Stage 1・Stage 2 における gain 上位 20 件.

Cryptography (2026). https://www.whitehouse.gov/wp-content/uploads/2026/06/M-26-15-Execution-of-the-Migration-to-Post-Quantum-Cryptography.pdf.

[9] NCSC: Timelines for migration to post-quantum cryptography (2025). https://www.ncsc.gov.uk/sites/default/files/2026-08/Timelines-for-migration-to-post-quantum-cryptography.pdf.

[10] EU PQC Workstream: A Coordinated Implementation Roadmap for the Transition to Post-Quantum Cryptography (2025). https://ec.europa.eu/newsroom/dae/redirection/document/117507.

[11] 国家サイバー統括室: 政府機関等における耐量子計算機暗号 (PQC) への移行について (中間とりまとめ) (案) (2025). https://www.cas.go.jp/jp/seisaku/pqc/dai2/shiryo1.pdf.

[12] OWASP: Authoritative Guide to CBOM (2025). https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf.

[13] Yamamuro, H., Uemura, S. and Fukushima, K.: SoK: Challenges for Implementing Automated Cryptography Discovery and Inventory Tool, *Proceedings of the 12th International Conference on Information Systems Security and Privacy - Volume 2: ICISSP*, INSTICC, SciTePress, pp. 157–168 (online), DOI: 10.5220/0014227100004061 (2026).

[14] Rattanavipanon, N., Suaboot, J. and Werapun, W.: A Toolchain for Assisting Migration of Software Executables Towards Post-Quantum Cryptography, *IEEE Access*, Vol. 13, pp. 4368–4380 (online), DOI: 10.1109/ACCESS.2024.3524309 (2025).

[15] Kim, H.-G., Lee, S.-W., Bang, J.-W., Kim, U.-J., Kang, H.-J., Kim, M.-S. and Seo, H.-J.: QED-Lite: Lightweight Detection of Quantum-Vulnerable ELF Binaries via Cryptographic Library Version Fingerprinting, Cryptology ePrint Archive, Paper 2026/660 (2026).

[16] Meijer, C., Moonsamy, V. and Wetzels, J.: Where's Crypto?: Automated Identification and Classification of Proprietary Cryptographic Primitives in Binary Code, *30th USENIX Security Symposium (USENIX Security 21)*, pp. 555–572 (online), available from ⟨https://www.usenix.org/conference/usenixsecurity21/⟩ (2021).

[17] Shang, X., Chen, G., Cheng, S., Guo, S., Zhang, Y., Zhang, W. and Yu, N.: FoC: Figure out the Cryptographic Functions in Stripped Binaries with LLMs, *ACM Trans. Softw. Eng. Methodol.*, (online), DOI: 10.1145/3731449 (2025). Just Accepted.

[18] Pacteau, A., Vitale, A., Balzarotti, D. and Aonzo, S.: Mnemocrypt: A Machine Learning Approach for Cryptographic Function Detection in x86 Executables, *Workshop on Binary Analysis Research*, (online), available from ⟨https://www.ndss-symposium.org/wp-content/uploads/bar2025-final2.pdf⟩ (2025).

[19] Hu, Y., He, Y., He, W., Li, H., Zhao, Y., Wang, S. and Gu, D.: Binary Cryptographic Function Identification via Similarity Analysis with Path-Insensitive Emulation, *Proc. ACM Program. Lang.*, Vol. 9, No. OOPSLA1 (online), DOI: 10.1145/3720415 (2025).

[20] Bos, J. W., Bronchain, O., Ducas, L., Fehr, S., Pornin, Y.-H. H. T., Postlethwaite, E. W., Prest, T., Pulles, L. N. and van Woerden, W.: HAWK (2025). https://hawk-sign.info/hawk-spec.pdf.

[21] Straznickas, Z. and Weis, S. A.: HAWK-n Key Recovery Reduces to SVP in Dimension $n/2+1$, Cryptology ePrint Archive, Paper 2026/1593 (2026).

[22] Bengen, H., Metz, J., Buehlmann, S., Alvarez, V. M. and Shields, W.: YARA: The pattern matching swiss knife for malware researchers (2026). https://github.com/VirusTotal/yara.

[23] Chen, T. and Guestrin, C.: XGBoost: A Scalable Tree Boosting System, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, New York, NY, USA, Association for Computing Machinery, p. 785–794 (online), DOI: 10.1145/2939672.2939785 (2016).

[24] Mallick, T., Kundu, A., Kompella, R. and Nita-Rotaru, C.: Classifying Implementations of Cryptographic Primitives and Protocols that Use Post-Quantum Algorithms, *Proceedings of the 31st ACM Symposium on Access Control Models and Technologies*, SACMAT '26, New York, NY, USA, Association for Computing Machinery, p. 137–148 (online), DOI: 10.1145/3750555.3811889 (2026).

[25] OpenSSL Project: OpenSSL. https://www.openssl.org/.

[26] Quynh, N. A.: Capstone: Next-Gen Disassembly Framework, Black Hat USA (2014).

© 2026 Information Processing Society of Japan